



BooleanVariable

Created time	July 5, 2026 5:17 AM
Category	CommonVariables
Last edited by	R.A R.A
Last updated time	July 5, 2026 7:06 AM

```
//true or false

bool isComplete = true;

//isComplete = !isComplete;

Console.WriteLine(!isComplete);
Console.WriteLine(isComplete);
```

משתנה בוליאני מכיל אחד משני ערכים, הוא אמת (**true**) או שקר (**false**).
בוליאני יכול להיות בשני מצבים שונים, האחד הוא **אמת** והשני הוא **שקר**, כאשר כברירת מחדל הוא **שקר**.
ה-**true** או **false** אלו שתי מילות מפתח שאנחנו יכולים להשתמש בהן.

בעולם פיתוח התוכנה המודרני, טיפוס הנתונים **Bool** (קיצור של **Boolean**) הוא הלב של קבלת ההחלטות במערכת.
אפשר להתייחס אליו כמו אל מתג הפעלה דיגיטלי: הוא חייב להיות או דלוק לחלוטין (**true**) או כבוי לחלוטין (**false**),
בלי שום תחום אפור או מצב ביניים.
זהו הטיפוס הקל ביותר בזיכרון, אך החשוב ביותר ללוגיקה.

ברירת המחדל בשפת #C

כאשר מכריזים על משתנה כזה במערכת ולא מכניסים לתוכו ערך באופן מיידי, סביבת הריצה של **Net** תגדיר אותו אוטומטית בתור **false**.
זהו מנגנון אבטחה וסדר שעוזר לנו למנוע שגיאות ומבטיח שהתוכנה תמיד מתחילה ממצב התחלתי סגור ובטוח, עד שאנחנו בתור המפתחים משנים אותו בצורה אקטיבית לאמת.

שימוש במילות המפתח

המילים **true** ו-**false** הן מילים שמורות ומוכרות היטב למנוע של השפה.

המשמעות היא שהקומפיילר מזהה אותן מיד, ולכן אי אפשר להשתמש בהן כשמות של משתנים, אלא אך ורק בתור הערכים המוחלטים שמייצגים אמת ושקר.

אנחנו נשתמש בהן כדי לבדוק האם תנאי מסוים התקיים, האם תהליך הסתיים בהצלחה, או האם למשתמש יש הרשאה לבצע פעולה.

~~~~~

משתנה בוליאני (**Boolean**) משמש במשפטי תנאי (**If statements**), מכיוון שבמשפט תנאי אנחנו שואלים "האם זה נכון?" או "האם זה לא נכון?".

זה כל הרעיון שעומד מאחורי משפט תנאי.

לכן, יש המון מקרים ב-**C#** שבהם אנחנו מעריכים מצבים כדי לקבוע אם הם נכונים (**true**) או לא נכונים (**false**).

המשמעות של ההסבר הזה היא שהבסיס לכל קבלת החלטות בקוד שלנו כמפתחי תוכנה נשען על משתנים בוליאניים. משפט תנאי מסוג **If** לא יכול לפעול על סמך מספרים או טקסטים בצורה גולמית, אלא הוא מחייב אותנו לספק לו ביטוי שהתוצאה הסופית שלו מצטמצמת לערך בוליאני: **אמת** או **שקר**.

כאשר אנחנו כותבים לוגיקה עסקית, אנחנו למעשה מייצרים ברקע שרשרת של שאלות "כן ולא".

לדוגמה, האם המשתמש סיים את ההרשמה? האם הנתון שנכנס למערכת תקין? התשובה לכל השאלות האלו נשמרת או מחושבת תמיד כמשתנה מסוג **Bool**.

הקשר בין משפט **If** למשתנה **Bool** הוא בלתי נפרד.

ברגע שמשפט ה-**If** מקבל לידיד את התשובה **true**, הוא מבין שעליו להריץ את בלוק הקוד ששייך לאותו תנאי.

לעומת זאת, אם הוא מקבל את התשובה **false**, הוא יודע שעליו לדלג על בלוק הקוד הזה ולהמשיך הלאה בהרצת התוכנית.

המשתנה הבוליאני הוא למעשה הרמזור שאומר למשפט התנאי מתי לעצור ומתי להמשיך.

### דוגמה להמחשת הרעיון:

אנחנו יכולים להגדיר משתנה בוליאני בשם **isRegistered** ולתת לו את הערך **true**.

לאחר מכן, נכתוב משפט **if** שבודק את המשתנה הזה.

התנאי יסתכל על הערך של **isRegistered**, יראה שהוא אכן **true**, ובתגובה יריץ את הפקודות ששמנו בתוך סוגרי התנאי.

~~~~~

```
bool isComplete = true;
isComplete = !isComplete;
```

למרות שקבענו שהערך הוא אמת (**true**):

```
bool isComplete = true;
```

על ידי הצבת תו ה"לא" (**סימן הקריאה !**) לפני המשתנה **isComplete**, אנחנו הופכים את הצהרת האמת להצהרת שקר (**false**):

```
isComplete = !isComplete;
```

לכן, אנחנו יכולים להשתמש בתו ה"לא" (!) כדי לבטא את ההפך ממה שהמשתנה הבוליאני מכיל כרגע, וזה יכול להיות מאוד שימושי.

זה גם קצת מתקיל, מכיוון שקל מאוד ללכת לאיבוד כשקוראים את המילה `isComplete!`, ואנחנו עלולים לפספס את העובדה שיש סימן קריאה (!) ממש בהתחלה.

אבל, זהו שימוש די נפוץ לסימן הקריאה (!), וכך בעצם אנחנו הופכים משתנה בוליאני.

היפוך משתנה בוליאני

בשפת **C#**, הסימן ! נקרא אופרטור השלילה הלוגית (**Logical NOT**).

התפקיד שלו הוא לקחת את הערך הנוכחי של המשתנה הבוליאני ולהפוך אותו בדיוק לצד השני, ממש כמו מתג של תאורה.

אם המשתנה כרגע מכיל `true`, הוספת סימן הקריאה לפניו תחזיר `false`, ולהפך.

בתעשייה המודרנית, פעולת ההיפוך הזו משמשת אותנו המון.

אנחנו משתמשים בזה למשל כשאנחנו רוצים לשנות מצב הלוך ושוב (**פעולה שנקראת Toggle**), או כשכותבים תנאים שליליים ברורים, כמו לבדוק אם תהליך מסוים טרם הסתיים כדי להמשיך להציג אנימציות טעינה.

הנקודה המרכזית שההסבר באנגלית ניסה להעביר היא חשיבות קריאות הקוד.

בסביבת פיתוח מתקדמת שבה קוראים מאות שורות קוד ביום, קל מאוד לרפרף עם העיניים ולפספס את סימן הקריאה הקטן הזה.

הפספוס הזה משנה לחלוטין את הלוגיקה של התוכנית ויכול לגרום לבאגים, ולכן נדרשת תשומת לב מיוחדת כשקוראים קוד שכולל שלילה לוגית.

~~~~~

```
bool isComplete = true;
Console.WriteLine(!isComplete);
Console.WriteLine(isComplete);
```

**אם נשים בתוך פקודת ההדפסה Console.WriteLine סימן קריאה (!):** זה אומר שזה "לא זה" (ההפך), ולכן נראה שהערך הוא `false`, כך שנוכל לכתוב את זה ישירות בשורת ההדפסה.

כדי להיות ברורים, אם נכתוב את הקוד כך ונריץ בקונסולה, אנחנו הולכים לקבל בפלט `false` ולאחר מכן `true`.

הסיבה לכך היא שלא באמת שינינו את המשתנה `isComplete` כשהוספנו לו סימן קריאה בתוך פקודת ההדפסה הראשונה.

אנחנו רק משנים את הפלט שמשמש במשתנה `isComplete`.

לכן, המשתנה המקורי עדיין מחזיק בדיוק באותו הערך, ובשורת ההדפסה השנייה הוא יודפס כ-`true` שוב, כי הוא תמיד נשאר `true` והוא מעולם לא שונה.

פשוט השתמשנו בערך שונה באופן זמני בהדפסה הראשונה על ידי הפיכת התוצאה.

ולכן שורת ההדפסה השנייה תדפיס `true`.

## אופרטור השלילה הלוגית

הקונספט כאן הוא יסוד חשוב מאוד בפיתוח תוכנה מודרני בתעשייה של שנת 2026, והוא ההבדל בין קריאת מצב לבין שינוי מצב.

סימן הקריאה (!) ב**C#** נקרא אופרטור שלילה.

התפקיד שלו הוא לקחת ערך בוליאני קיים ולהחזיר את ההפך שלו, אבל הוא עושה זאת על ידי יצירת תוצאה חדשה וזמנית באותו רגע נתון, מבלי לגעת במידע המקורי ששמור בזיכרון של המחשב.

## ההבדל בין הערכה להשמה

כאשר מפעילים את סימן הקריאה בתוך פקודת ההדפסה, המערכת רק מעריכה את הביטוי כדי להציג אותו על המסך. היא רואה שהמשתנה המקורי הוא **true**, מחשבת שההפך שלו הוא **false**, ומדפיסה **false**.

מיד לאחר מכן, הערך ההפוך הזה נמחק מהזיכרון הזמני.

המשתנה **isComplete** שישב בזיכרון הראשי מעולם לא עודכן.

בתכנות מתקדם אנחנו מקפידים מאוד על ההפרדה הזו כדי למנוע באגים של שינוי מידע בטעות.

כדי שמשתנה באמת ישתנה, חובה להשתמש באופרטור ההשמה, כלומר בסימן שווה (=).

כל עוד לא כתבנו במפורש שווה (=) כדי להכניס ערך חדש לתוך המשתנה, הוא תמיד יישאר בדיוק כפי שהגדרנו אותו בהתחלה.